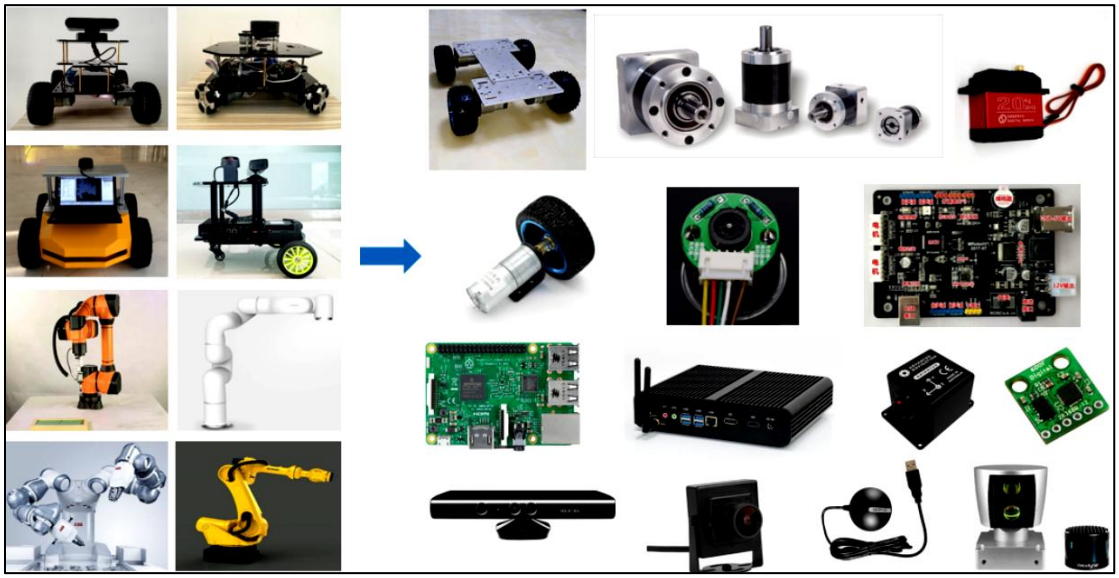
17.ROS2 URDF 机器人建模方法

1.URDF 简介

ROS 中的建模方法叫做 URDF，全称是统一机器人描述格式，不仅可以清晰描述机器人自身的模型，还可以描述机器人的外部环境。URDF 模型文件使用的是 XML 格式。

2.机器人的组成

建模描述机器人的过程中，我们自己需要先熟悉机器人的组成和参数，比如机器人一般是由硬件结构、驱动系统、传感器系统、控制系统四大部分组成，市面上一些常见的机器人，无论是移动机器人还是机械臂，我们都可以按照这四大组成部分进行分解。

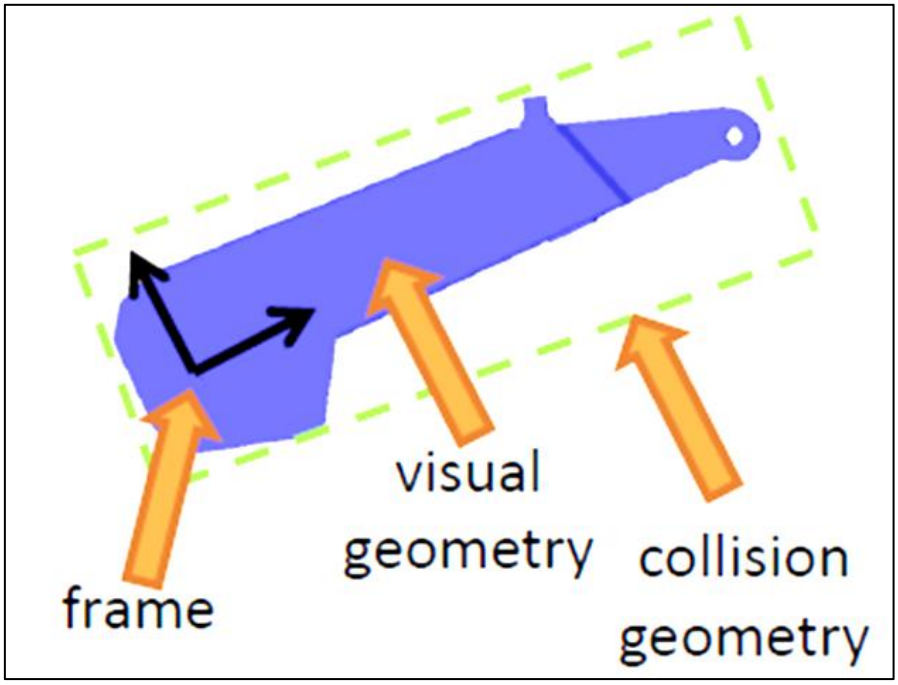


- 硬件结构就是底盘、外壳、电机等实打实可以看到的设备；
- 驱动系统就是可以驱使这些设备正常使用的装置，比如电机的驱动器、电源管理系统等；
- 传感系统包括电机上的编码器、板载的 IMU、安装的摄像头、雷达等等，便于机器人感知自己的状态和外部的环境；
- 控制系统就是我们开发过程的主要载体了，一般是树莓派、电脑等计算平台，以及里边的操作系统和应用软件。

3.URDF 语法

3.1 连杆 Link 的描述

标签用来描述机器人某个刚体部分的外观和物理属性，外观包括尺寸、颜色、形状，物理属性包括质量、惯性矩阵、碰撞参数等。



以这个机械臂连杆为例，它的 link 描述如下：

```
<link name="link 4">
    <visual>
        <geometry>
            <mesh filename="link 4.stl"/>
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
    </visual>
    <collision>
```

```
    <geometry>
        <cylinder length="0.5" radius="0.1"/>
    </geometry>
    <origin xyz="0 0 -0.05" rpy="0 0 0" />
    </collision>
</link>
```

link 标签中的 name 表示该连杆的名称，我们可以自定义，未来 joint 连接 link 的时候，会使用到这个名称。

link 里边的<visual>部分用来描述机器人的外观，比如：

- <geometry>表示几何形状，里边使用<mesh>调用了一个在三维软件中提前设计好的蓝色外观，就是这个 stl 文件，看上去和真实机器人是一致的。
- <origin>表示坐标系相对初始位置的偏移，分别是 x、y、z 方向上的平移，和 roll、pitch、raw 旋转，不需要偏移的话，就全为 0。

第二部分<collision>，描述碰撞参数，里边的内容似乎和<visual>一样，也有<geometry>和<origin>，看似相同，其实区别还是比较大的。

- <visual>部分重在描述机器人看上去的状态，也就是视觉效果。
- <collision>部分则是描述机器人运动过程中的状态,比如机器人与外界如何接触算作碰撞。

3.2 关节 joint 描述

机器人模型中的刚体最终要通过关节 joint 连接之后，才能产生相对运动。

URDF 中的关节有六种运动类型。

1） continuous，描述旋转运动，可以围绕某一个轴无限旋转，比如小车的轮子，就属于这种类型。

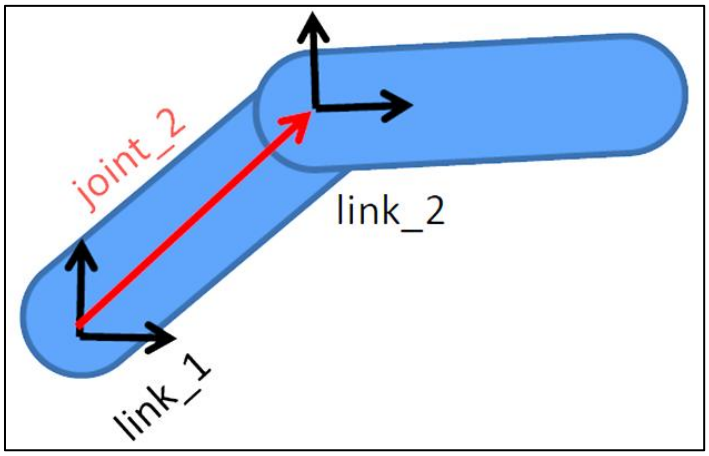
关节类型	描述
continuous	旋转关节，可以围绕单轴无限旋转
revolute	旋转关节，类似于 continuous，但是有旋转的角度极限
prismatic	滑动关节，沿某一轴线移动的关节，带有位置极限
fixed	固定关节，不允许运动的特殊关节
floating	浮动关节，允许进行平移、旋转运动
planar	平面关节，允许在平面正交方向上平移或者旋转

2） revolute，也是旋转关节，和 continuous 类型的区别在于不能无限旋转，而是带有角度限制，比如机械臂的两个连杆，就属于这种运动。

3） prismatic，是滑动关节，可以沿某一个轴平移，也带有位置的极限，一般直线电机就是这种运动方式。

4） fixed，固定关节，是唯一一种不允许运动的关节，不过使用还是比较频繁的，比如相机这个连杆，安装在机器人上，相对位置是不会变化的，此时使用的连接方式就是 Fixed。

5） Floating 是浮动关节，第六种 planar 是平面关节，这两种使用相对较少。



在 URDF 模型中，每一个 link 都使用这样一段 xml 内容描述，比如关节的名字叫什么，运动类型是哪一种。

```
<joint name="joint 2" type="revoluteu">
    <parent link=link 1"/>
    <child link=ulink 2"/>
    <origin xyz="0.2 0.2 0" rpy="0 0 0"/>
    <axis xyz="0 0 1"/>
    <limit lower="-3.14" upper="3.14" velocity="1.0"/>
</joint>
```

- parent 标签：描述父连杆；
- child 标签：描述子连杆，子连杆会相对父连杆发生运动；
- origin：表示两个连杆坐标系之间的关系，也就是图中红色的向量，可以理解为这两个连杆该如何安装到一起；
- axis 表示关节运动轴的单位向量，比如 z 等于 1，就表示这个旋转运动是围绕 z 轴的正方向进行的；
- limit 就表示运动的一些限制了，比如最小位置，最大位置，和最大速度等。

注意：ROS 中关于平移的默认单位是 m，旋转是弧度（不是度），所以这里的 3.14 就

表示可以在-180 度到 180 度之间运动，线速度是 m/s，角速度是 rad/s。

4.完整机器人模型

4.1 创建机器人模型

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/**”指令，切换到 ros2_ws 工作空间的 src 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/
ubuntu@ubuntu:~/ros2_ws/src$
```

2） 输入“**ros2 pkg create urdf_demo --build-type ament_python --dependencies rclpy**”并按下回车，创建一个名为“**urdf_demo**”的功能包，添加依赖关系 rclpy。

```
ubuntu@ubuntu:~/ros2_ws/src$ ros2 pkg create urdf_demo --build-type ament_python --dependencies rclpy
going to create a new package
package name: urdf_demo
destination directory: /home/ubuntu/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['ubuntu <ubuntu@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_python
dependencies: ['rclpy']
creating folder ./urdf_demo
creating ./urdf_demo/package.xml
creating source folder
creating folder ./urdf_demo/urdf_demo
creating ./urdf_demo/setup.py
creating ./urdf_demo/setup.cfg
creating folder ./urdf_demo/resource
creating ./urdf_demo/resource/urdf_demo
creating ./urdf_demo/urdf_demo/__init__.py
creating folder ./urdf_demo/test
creating ./urdf_demo/test/test_copyright.py
creating ./urdf_demo/test/test_flake8.py
creating ./urdf_demo/test/test_pep257.py
```

3） 输入“**cd urdf_demo/**”指令，切换到 urdf_demo 功能包下。

```
ubuntu@ubuntu:~/ros2_ws/src$ cd urdf_demo/
ubuntu@ubuntu:~/ros2_ws/src/urdf_demo$
```

4） 输入“**mkdir urdf**”指令，创建 urdf 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/urdf_demo$ mkdir urdf
ubuntu@ubuntu:~/ros2_ws/src/urdf_demo$
```

5） 输入“**cd urdf/**”指令，切换到 urdf 文件夹下。

```
ubuntu@ubuntu:~/ros2_ws/src/urdf_demo$ cd urdf/
ubuntu@ubuntu:~/ros2_ws/src/urdf_demo/urdf$
```

1） 输入指令“**gedit simple_demo.urdf**”编辑程序，复制下面程序,使用快捷键

“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/urdf_demo/urdf$ gedit simple_demo.urdf
```

```
<?xml version="1.0"?>

<robot name="materials">



  <material name="blue">

    <color rgba="0 0 0.8 1"/>

  </material>



  <material name="white">

    <color rgba="1 1 1 1"/>

  </material>







  <link name="base_link">

    <visual>

      <geometry>

        <cylinder length="0.6" radius="0.2"/>

      </geometry>

      <material name="blue"/>

    </visual>

  </link>
```

```
<link name="right_leg">
  <visual>
    <geometry>
      <box size="0.6 0.1 0.2"/>
    </geometry>
    <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
    <material name="white"/>
  </visual>
</link>

<joint name="base_to_right_leg" type="fixed">
  <parent link="base_link"/>
  <child link="right_leg"/>
  <origin xyz="0 -0.22 0.25"/>
</joint>

<link name="left_leg">
  <visual>
    <geometry>
      <box size="0.6 0.1 0.2"/>
    </geometry>
```

```
    <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
    <material name="white"/>
  </visual>
 </link>

 <joint name="base_to_left_leg" type="fixed">
  <parent link="base_link"/>
  <child link="left_leg"/>
  <origin xyz="0 0.22 0.25"/>
 </joint>

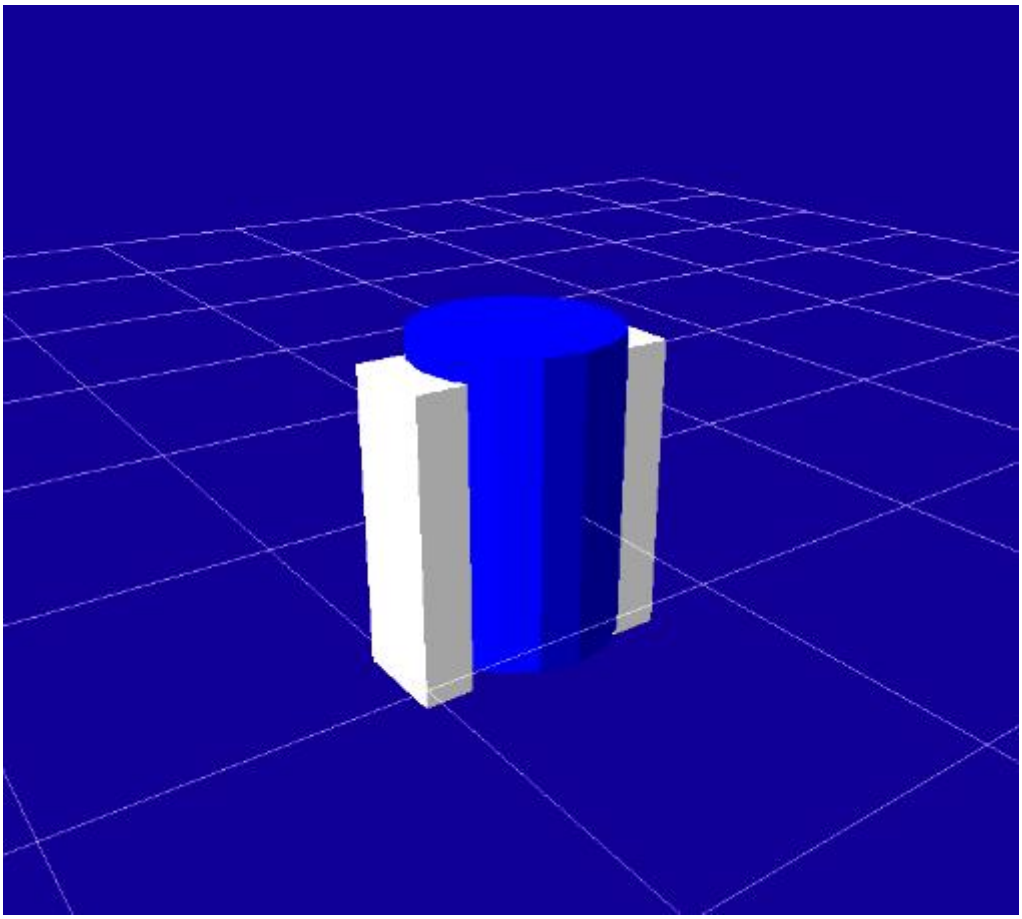
</robot>
```

6) 输入指令“**chmod +x simple_demo.urdf**”，并按下回车，为保存的 simple_demo.urdf 赋予可执行权限。

```
ubuntu@raspberrypi:~/hiwonder_ws/src/urdf_demo/urdf$ chmod +x simple_demo.urdf
ubuntu@raspberrypi:~/hiwonder_ws/src/urdf_demo/urdf$
```

4.2 机器人模型加载

模型的效果如下图所示，后面的课程当中将会给大家讲解如何加载到一个可视化界面当中。



定义了一个简单的机器人模型,主要组成和功能如下:

1) 定义了两个材质 - 蓝色和白色。

```
<?xml version="1.0"?>
<robot name="materials">

  <material name="blue">
    <color rgba="0 0 0.8 1"/>
  </material>

  <material name="white">
    <color rgba="1 1 1 1"/>
  </material>
```

2) 定义了基座连杆(base_link),使用蓝色材质和圆柱体几何.

```
<link name="base_link">
  <visual>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
    <material name="blue"/>
  </visual>
</link>
```

3) 定义了右腿和左腿连杆,使用白色材质和箱体几何。

```
<link name="right_leg">
  <visual>
    <geometry>
      <box size="0.6 0.1 0.2"/>
    </geometry>
    <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
    <material name="white"/>
  </visual>
</link>
```

```
<link name="left_leg">
  <visual>
    <geometry>
      <box size="0.6 0.1 0.2"/>
    </geometry>
    <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
    <material name="white"/>
  </visual>
</link>
```

4) 为右腿和左腿定义了固定连接的转动关节,分别连接到基座连杆右腿连接的原点位于(0, -0.22, 0.25),左腿位于(0, 0.22, 0.25),腿部箱体设置了旋转角度,使其竖立直立。

```
<joint name="base_to_right_leg" type="fixed">
  <parent link="base_link"/>
  <child link="right_leg"/>
  <origin xyz="0 -0.22 0.25"/>
</joint>
```

```
<joint name="base_to_left_leg" type="fixed">
  <parent link="base_link"/>
  <child link="left_leg"/>
  <origin xyz="0 0.22 0.25"/>
</joint>
```

主要功能是定义了这个简单机器人的材质、几何形状、连杆及其连接关系。

通过定义材质、连杆、视觉组件、关节等基本语法,完成了这个机器人模型的定义。